

Mini-Challenge: KI-gestütztes Programmieren

Version 1.1

Dieses Dokument beschreibt den schriftlichen Leistungsnachweis des KIP-Moduls.

Übersicht

Die Mini-Challenge ist der zentrale Leistungsnachweis dieses Moduls und soll Dir ermöglichen, die Lernergebnisse LE1, LE2 und LE3 in einem anspruchsvollen, praxisnahen Projekt anzuwenden und zu vertiefen. Du wirst ein ambitioniertes Software-, Data Science- oder KI-Projekt entwickeln, das auch mit KI-Unterstützung herausfordernd bleibt.

Wichtig: Das Ziel ist nicht primär ein funktionierendes Endprodukt, sondern insbesondere die Reflexion und Dokumentation Deines Entwicklungsprozesses, Deiner Strategien und Deiner Lernerfahrungen im Umgang mit KI-gestützten Entwicklungstools.

Zeitplan

- **SW 6:** Einreichung des Exposés via Mail, Besprechung davon in der Kontaktstunde (unbenotet, formatives Feedback)
- **SW 2-11:** Projektentwicklung
- **SW 11, Sonntag 18:00 Uhr:** Abgabe (Repo + Entwicklungsbericht)

Teil 1: Exposé (Abgabe SW 6)

Zweck

Das Exposé dient dazu, Dein Projekt frühzeitig zu planen, das Anforderungsszenario zu definieren und sicherzustellen, dass die Komplexität angemessen ist. Du erhältst formatives Feedback, um Dein Projekt bei Bedarf anzupassen.

Anforderungen an das Exposé (1-2 Seiten)

1. **Projekttitel und Kurzbeschreibung**
 - Prägnanter Titel
 - Zusammenfassung in 3-4 Sätzen
2. **Projektbeschreibung**

- Was soll entwickelt werden?
- Welches Problem wird gelöst?
- Welche Technologien werden voraussichtlich eingesetzt?

3. Anforderungsszenario nach LE1

Verorte Dein Projekt auf folgenden Dimensionen:

- Dein Vorwissen zur Lösung (sehr vertraut bis unbekannt)
- Notwendigkeit/Wunsch des Verständnisses der relevanten Elemente (im Detail - nein)
- Cybersecurity-Bedrohungslevel (öffentlich unkritisch - kritisch)
- Mögliche versteckte Konsequenzen von Bugs? (ja - vielleicht - nein)
- Konsequenzen potenzieller Probleme (unkritisch - schädlich)
- Haftung/Verantwortung (ja - nein)

4. Begründung der Komplexität

- Warum ist dieses Projekt auch mit KI-Unterstützung herausfordernd?
- Welche spezifischen Herausforderungen erwartest Du?
- Welche Aspekte könnten die KI-Tools an ihre Grenzen bringen?

5. Geplante Funktionalitäten (Minimum Viable Product und Stretchgoals)

- Welche Kernfunktionalitäten müssen auf jeden Fall umgesetzt werden?
- Welche zusätzlichen Features sind "nice to have" als Stretchgoals, falls die Implementierung schneller geht als erwartet?

6. Erste Überlegungen zur Entwicklungsstrategie

- Wie planst Du vorzugehen?
- Ist es sinnvoll Methoden des Test-Driven-Development zu verwenden?
- Wie willst Du Verständnis und Ownership sicherstellen?

Kriterien für ein geeignetes Projekt

Dein Projekt sollte folgende Eigenschaften aufweisen:

Angemessene Komplexität: Das Projekt sollte mehrere technische Herausforderungen beinhalten (z.B. Integration verschiedener APIs, komplexe Datenverarbeitung, State Management, Deployment, etc.).

Lernpotenzial: Du solltest in mindestens einem relevanten Bereich Neuland betreten (neue Technologie, Framework, Problemdomäne).

KI-Grenzen testen: Das Projekt sollte Aspekte enthalten, bei denen KI-Tools voraussichtlich Schwierigkeiten haben (z.B. komplexe Architekturen, domänenspezifisches Wissen, Performance-Optimierung, Fehlersuche in komplexen Systemen)

Umfang: Realistisch in der gegebenen Zeit umsetzbar, aber anspruchsvoll genug, dass kontinuierliche Arbeit nötig ist.

Teil 2: Projektentwicklung (SW 2-11)

Während der Entwicklung solltest Du:

1. Kontinuierlich dokumentieren:

- Führe ein Entwicklungstagebuch. Nutze, dass Du die Promptdialoge exportieren kannst.
- Screenshoote wichtige Momente, falls nötig.
- Notiere Deine Strategiewechsel und deren Gründe

2. Reflektiert vorgehen:

- Überprüfe regelmässig Dein Anforderungsszenario
- Passe Deine Strategie an, wenn nötig
- Dokumentiere Erfolge und Misserfolge

3. Verständnis aufbauen:

- Versuche aktiv, den generierten Code zu verstehen, besonders bei Kernkomponenten.
- Implementiere kritische Teile selbst oder mit minimaler KI-Hilfe.
- Überlege Dir, falls Du Test-Driven-Development-Methoden benutzt, ob die Tests sinnvoll Fehlerbilder abfangen.

Teil 3: Abgabe (SW 11, Sonntag 18:00 Uhr)

Lieferobjekte

1. Git-Repository

- Vollständiger, funktionsfähiger Code
- README mit Setup-Anleitung und Projekt-Übersicht
- Saubere Commit-History (sinnvolle Commit-Messages)
- Ggf. Requirements/Dependencies-Datei

2. Entwicklungsbericht (8-12 Seiten)

Struktur des Entwicklungsberichts

1. Einleitung

- Kurze Projektbeschreibung.
- Überblick über die entwickelte Lösung .
- Ergebnis in Bezug auf die geplanten Ziele. Was konnte nicht umgesetzt werden?

2. Anforderungsszenario und dessen Entwicklung – LE1

- Initiales Anforderungsszenario (aus dem Exposé)
- **Hat sich das Anwendungsszenario während der Entwicklung verschoben?**
 - Wenn ja: Wie und warum?
 - Wie hast Du darauf reagiert?
 - Welche Dimensionen haben sich verändert (Komplexität, Verständnisanspruch, Sicherheit, etc.)?
- Finale Einordnung des Projekts
- Reflexion: War die initiale Einschätzung korrekt?

3. Entwicklungsstrategie und KI-Einsatz – LE2

3.1 Durchgeführtes Vorgehen mit Beispielen

- Wie bist Du konkret vorgegangen? (Mit spezifischen Beispielen)
- Welche KI-Funktionen hast Du genutzt? (Chat, Code-Completion, Refactoring, etc.)
- Wie hast Du Deine Prompts strukturiert?

3.2 Was hat funktioniert?

- Welche Ansätze/Strategien waren erfolgreich?
- Konkrete Beispiele mit Screenshots oder Code-Ausschnitten

3.3 Wo musste das Vorgehen angepasst werden?

- An welchen Punkten gab es Probleme, die Strategiewechsel nötig gemacht haben?
- Warum hast Du das Vorgehen umgestellt?
- Was war das Ergebnis der Anpassung?

3.4 Was hat nicht funktioniert?

- Wo bist Du auf Grenzen der KI-Tools gestoßen, die Du nicht überwinden konntest?

- Welche Probleme konnten nicht gelöst werden oder nur manuell?
- Kannst Du die Gründe für die Probleme abschätzen (Modellgrenzen, Kontextprobleme, Komplexität, etc.)?

3.5 Umgang mit Modellgrenzen

- Wie hast Du Limitierungen erkannt?
- Welche Strategien hast Du entwickelt, um damit umzugehen?
- Wann hast Du entschieden, ohne KI-Unterstützung weiterzuarbeiten?

4. Verständnis und Ownership – LE3

4.1 Verständnistiefe der verschiedenen Komponenten

Beschreibe für die Hauptkomponenten Deiner Lösung:

- Welche Teile verstehst Du im Detail?
- Welche Teile verstehst Du konzeptuell?
- Welche Teile sind für Dich eine "Black Box"?

Dies könnte z. B. folgendermassen beschrieben werden:

Komponente	Verständnistiefe	Begründung
Datengenerierungslogik und Augmentierung	Vollständig	Selbst entworfen und durchoptimiert
API-Endpunkte	Größtenteils	KI-generiert, aber nachvollzogen und getestet
WebSocket-Logik	Konzeptuell	Verstehe das Konzept, aber Details unklar
Deployment-Pipeline	Oberflächlich	Von KI generiert, läuft aber

4.2 Methoden zur Herstellung von Verständnis

- Welche Strategien hast Du angewendet, um für Dich relevanten Code zu verstehen?
 - Code-Reviews, Debugging, selberschreiben von Test für TDD ,Umschreiben, Dokumentation lesen, etc.
- Welche haben gut funktioniert?
- Wo hast Du bewusst auf tiefes Verständnis verzichtet? Warum?

4.3 Ownership und Verantwortung

- Fühlst Du Dich als "Owner" der Lösung?
- Könntest Du die Lösung erklären, warten und manuell weiterentwickeln?
- Wo würdest Du bei Problemen unsicher sein?

4.4 Kritische Reflexion

- **Bereust Du es, bestimmte Teile nicht besser zu verstehen?**
 - Wenn ja: Welche und warum?
 - Was wären die Konsequenzen, wenn Du diese Teile verstehen müsstest?
- Wo war der Trade-off zwischen Geschwindigkeit und Verständnis gerechtfertigt?
- Wo würdest Du im Nachhinein anders vorgehen?

4.5 Risiken und Sicherheit

- Welche Risiken ergeben sich aus mangelndem Verständnis?
- Welche Cybersecurity-Aspekte hast Du berücksichtigt?
- Wie hast Du sichergestellt, dass keine sensiblen Daten an die KI übermittelt wurden?

5. Qualitätssicherung

Wie hast Du sichergestellt, dass die Anforderungen erfüllt wurden?

- Welche Teststrategien hast Du angewendet?
- Wie hast Du die Funktionalität validiert?
- Welche Quality-Assurance-Maßnahmen wurden durchgeführt?

6. Fazit und Lessons Learned

- Wichtigste Erkenntnisse aus dem Projekt
- Was würdest Du beim nächsten Projekt anders machen?
- Wie hat sich Deine Perspektive auf KI-gestützte Entwicklung verändert?
- Was sind Deine Empfehlungen für andere Studis, die demnächst mit KIP beginnen.

Anhänge (optional, nicht im Seitenumfang)

- Screenshots von relevanten KI-Interaktionen
- Code und Promptdialog-Beispiele, die im Text referenziert werden
- Entwicklungstagebuch-Auszüge

Bewertungskriterien

Die Bewertung erfolgt anhand folgender Kriterien:

1. Verständlichkeit des Entwicklungsbericht (25%)

- Ist der Bericht klar strukturiert und gut lesbar?
- Sind die Argumentationen nachvollziehbar?
- Werden konkrete Beispiele gegeben?
- Ist die Dokumentation vollständig und konsistent?

Bewertungsstufen:

- **5.5-6.0:** Hervorragend strukturiert, sehr klar geschrieben, reichlich konkrete Beispiele, vollständig
- **5.0-5.4:** Gut strukturiert, verständlich, einige gute Beispiele, weitgehend vollständig
- **4.5-4.9:** Grundstruktur vorhanden, stellenweise unklar, wenige Beispiele, kleine Lücken
- **4.0-4.4:** Schwache Struktur, oft unklar, kaum Beispiele, relevante Lücken
- **<4.0:** Unstrukturiert, unverständlich, keine Beispiele, unvollständig

2. Reflexion des Anforderungsszenarios (20%)

- Wurde das Projekt initial korrekt eingeordnet?
- Wurden Veränderungen des Szenarios erkannt und reflektiert?
- Wurde die Strategie an das Szenario angepasst?
- Ist die Reflexion stichhaltig und kritisch?

Bewertungsstufen:

- **5.5-6.0:** Differenzierte initiale Einordnung, Veränderungen wurden erkannt und analysiert, strategische Anpassungen dokumentiert, tiefgründige Reflexion
- **5.0-5.4:** Gute initiale Einordnung, wesentliche Veränderungen erkannt, Anpassungen vorgenommen, gute Reflexion
- **4.5-4.9:** Grundlegende Einordnung, einige Veränderungen erkannt, teilweise angepasst, oberflächliche Reflexion
- **4.0-4.4:** Ungenaue Einordnung, kaum Veränderungen reflektiert, wenig Anpassung, schwache Reflexion
- **<4.0:** Keine klare Einordnung, keine Reflexion von Veränderungen, keine Anpassung

3. Entwicklungsstrategie und agile Anpassung (25%)

- Wurde eine durchdachte initiale Strategie entwickelt?
- Wurden KI-Tools sinnvoll und reflektiert eingesetzt?
- Wurde die Strategie bei Bedarf angepasst?
- Wurden Erfolge und Misserfolge differenziert dokumentiert?
- Wurde mit Modellgrenzen konstruktiv umgegangen?

Bewertungsstufen:

- **5.5-6.0:** Exzellente initiale Strategie, sehr reflektierter KI-Einsatz, agile Anpassungen mit klarer Begründung, hervorragende Dokumentation von Erfolgen/Misserfolgen, kreative Lösungen für Toolgrenzen
- **5.0-5.4:** Gute Strategie, sinnvoller KI-Einsatz, nachvollziehbare Anpassungen, gute Dokumentation, konstruktiver Umgang mit Grenzen
- **4.5-4.9:** Basis-Strategie, grundlegender KI-Einsatz, einige Anpassungen, unvollständige Dokumentation, Grenzen erkannt
- **4.0-4.4:** Vage Strategie, unreflektierter KI-Einsatz, kaum Anpassungen, schwache Dokumentation, Grenzen nicht adressiert
- **<4.0:** Keine erkennbare Strategie, wahlloser KI-Einsatz, keine Anpassungen, keine Dokumentation

4. Reflexion von Verständnis, Ownership und Schwächen (25%)

- Wird das eigene Verständnis differenziert und ehrlich dargestellt?
- Wurden Methoden zur Verständnisbildung angewendet?
- Wird reflektiert, wo mangelndes Verständnis problematisch ist?
- Werden Schwächen der Lösung und des Vorgehens kritisch benannt?
- Ist die Selbstreflexion authentisch und tiefgehend?

Bewertungsstufen:

- **5.5-6.0:** Sehr differenzierte Darstellung des Verständnisses, vielfältige Methoden angewendet, kritische und ehrliche Reflexion von Lücken und Konsequenzen, authentische Selbstkritik, klare Einsichten
- **5.0-5.4:** Gute Differenzierung, einige Methoden genutzt, ehrliche Reflexion, Schwächen benannt, gute Selbstreflexion
- **4.5-4.9:** Grundlegende Darstellung, wenige Methoden, oberflächliche Reflexion, einige Schwächen erkannt, begrenzte Selbstkritik

- **4.0-4.4:** Undifferenzierte Darstellung, kaum Methoden, kaum Reflexion, Schwächen nicht erkannt, keine echte Selbstkritik
- **<4.0:** Keine Differenzierung, keine Methoden, keine Reflexion, keine Schwächen benannt

5. Qualitätssicherung und Anforderungserfüllung (5%)

- Wurden angemessene QA-Maßnahmen durchgeführt?
- Wurde systematisch getestet?
- Wurde validiert, dass Anforderungen erfüllt sind?

Bewertungsstufen:

- **5.5-6.0:** Systematische QA mit verschiedenen Methoden, umfassende Tests, klare Validierung
- **5.0-5.4:** Gute QA-Maßnahmen, ausreichende Tests, nachvollziehbare Validierung
- **4.5-4.9:** Basis-QA, grundlegende Tests, teilweise Validierung
- **4.0-4.4:** Minimale QA, kaum Tests, unklare Validierung
- **<4.0:** Keine erkennbare QA, keine Tests, keine Validierung

Zusätzliche Bewertungsdimensionen:

Projektdurchführung (nicht separat bewertet, fließt in die Kriterien ein):

- Ist das Repository professionell strukturiert, dokumentiert inkl. Commithistory?

Wichtig: Es geht nicht um Perfektion der Lösung, sondern um die Reflexion und Dokumentation des Prozesses. Auch ein Projekt, das nicht alle Ziele erreicht hat, kann eine sehr gute Bewertung erhalten, wenn der Prozess hervorragend dokumentiert und reflektiert wurde.

Tipps für eine erfolgreiche Bearbeitung

1. **Dokumentiere kontinuierlich:** Führe ein Entwicklungstagebuch. Es ist schwer, sich nach wenigen Tagen an Details zu erinnern.
2. **Screenshots sind für die Illustration hilfreich:** Speichere relevante KI-Interaktionen ab, besonders bei Grenzen oder interessanten Strategien.
3. **Sei ehrlich:** Authentische Reflexion ist wertvoller als beschönigtes "alles lief perfekt". Scheitern ist ein nötiger Teil des Lernprozesses.
4. **Konkret, nicht abstrakt:** Nutze konkrete Beispiele aus Deinem Projekt statt abstrakter Aussagen.

5. **Balance zwischen Entwicklung und Reflexion:** Plan ausreichend Zeit für das Schreiben des Berichts ein (ca. 1-2 Wochen).
6. **Bei Problemen: Nutze die Kontaktstunden:** Wenn Du feststeckst oder das Projekt nicht umsetzbar ist, sprich mich an.

Häufig gestellte Fragen (FAQ)

Q: Muss das Projekt perfekt funktionieren?

A: Nein. Ein funktionierendes MVP ist das Ziel, aber hervorragende Reflexion eines teilweise gescheiterten Projekts ist wertvoller als ein perfektes Projekt ohne Reflexion.

Q: Wie viel Code muss ich selbst schreiben?

A: Es gibt keine feste Quote. Wichtig ist, dass Du reflektierst, wann und warum Du KI nutzt oder nicht nutzt, und dass Du die Konsequenzen verstehst.

Q: Darf ich Code aus Open Source Projekten verwenden?

A: Ja, aber kennzeichne fremden Code klar und reflektiere, wie dies Dein Verständnis beeinflusst.

Q: Was, wenn sich herausstellt, dass mein Projekt zu einfach/schwer ist?

A: Passe es an! Dokumentiere diese Entscheidung im Bericht als Teil der agilen Anpassung und besprich es mit mir in der Kontaktstunde.

Q: Wie detailliert sollen die Code-Beispiele sein?

A: Zeige kurze, relevante Ausschnitte, die Deine Punkte illustrieren. Nicht die ganze Codebase im Bericht abladen.

Q: Muss ich alle KI-Chats dokumentieren?

A: Nein, fokussiere Dich auf die interessanten, lehrreichen oder problematischen Interaktionen.

Q: Welche Programmiersprache/Technologie muss ich verwenden?

A: Freie Wahl – wähle was zu Deinem Projekt passt. Die Reflexion des Prozesses ist wichtiger als die spezifische Technologie.

Abgabe

Format

- **Repository:** Git-Repository (GitHub, GitLab, oder als .zip-Archiv) Einladung zum Repo
- **Bericht:** PDF-Format

Checkliste vor der Abgabe

- ☐ Repository ist vollständig und funktionsfähig
 - ☐ README.md ist vorhanden mit Setup-Anleitung
 - ☐ Keine Secrets oder API-Keys im Code
 - ☐ Sinnvolle Commit-History
 - ☐ Bericht ist vollständig (alle Abschnitte vorhanden)
 - ☐ Bericht hat 8-12 Seiten (exkl. Anhänge)
 - ☐ PDF ist korrekt formatiert und lesbar
 - ☐ Beide Dateien (Repo-Link/Zip + Bericht-PDF) wurden eingereicht
-

Viel Erfolg bei der Mini-Challenge!